What aggregate data can and cannot tell us: synthetic evenings for testing Lyft-transfer rules on Blue Jay Night Ride

Team FourSight · EN.553.602 Research and Design in Mathematics: Data Mining · Midterm, October 2026 · DRAFT
Presenters: P1 (slides 1–4) · P2 (5–8) · P3 (9–12), roles to be confirmed

Blue Jay Night Ride, and what we proposed in September

The service

1 Request	a rider books a trip in the app (6 pm–2 am, Homewood area)
2 Shared van	a dispatcher assigns a van; other riders may share it
3 Curb to curb	picked up at the curb, dropped at the destination
or Lyft	Hopkins can send a Lyft instead; about 1 in 4 April riders (24.5%) rode Lyft

What we proposed

- Replay real service nights from trip-level records
- Compare dispatch methods: simple greedy rules vs an optimization model (a mixed-integer linear program, MILP) re-solved as requests arrive
- Treat Lyft as paid overflow chosen inside the optimizer
- Use the published service targets as hard constraints

Proposal numbers we later corrected

- "5,100 rides per hour at 6 pm" was a 30-day total: about 170 completed rides in one evening's 6 pm hour
- "28 vans" counted all services, not only Homewood
- "1 in 4 overflow to Lyft": a share of riders (people), not of requests or trips

Source: Proposal deck (Sep 2026); April 2026 JHU/TransLoc dashboard: 44,374 riders, van + Lyft, 24.5% by Lyft (transloc_operations.md).

How the research question changed

	September proposal	Now
Data	Trip-level records were expected	Only aggregate dashboards: totals, shares, percentiles
Question	Which dispatch algorithm is best?	Which Lyft-transfer rule, with the van dispatcher held fixed?
Lyft	Paid overflow inside the optimizer	A separate, explicit transfer rule; no money modelled (cost unknown)
Targets	Hard constraints	Wait targets are reported, not imposed; plans may not create a ride longer than max(25 min, 2 × the direct drive + 5 min)
Claims	About real nights	About synthetic evenings consistent with April's totals

Why switch: on synthetic nights, a ranking of dispatch algorithms would mostly reflect our guesses about demand detail. Replaying the same night with one lever changed (the transfer rule, the decision Hopkins' dispatchers actually make) isolates that lever's effect under stated assumptions.

Research question

With the van dispatcher held fixed, which rule for transferring waiting riders to Lyft gives the best rider service for the Lyft use it requires, and why?

The six transfer rules we compare

R0	never transfer
R1(10)	after 10 min of waiting
R1(15)	after 15 min of waiting
R1u(10)	after 10 min with no van assigned
R2(15)	if the predicted total wait > 15 min
R3(2)	if the van is predicted later than a Lyft (+2 min)

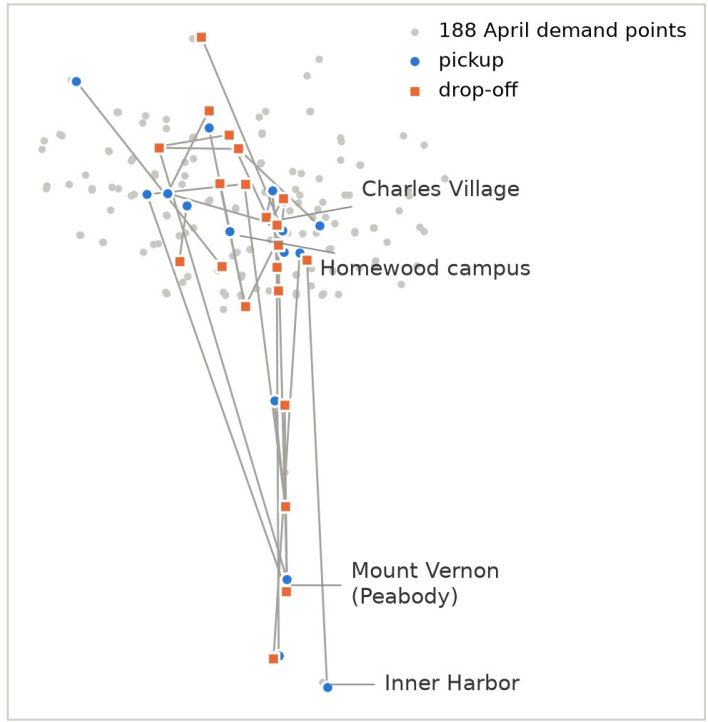
Source: Proposal deck; decision log D-23–D-27, D-45, D-49; research_review.md.

Building an evening from aggregate data

OBSERVED	DERIVED	UNKNOWN	CHOSEN
How many requests			
About 5,100 completed rides in the 6 pm hour over 30 days (tooltip: 5,098); 27.6 vehicles on duty (all services)	÷30 → 170 per evening; ×2.087 requests per completion → 355; ÷27.6 → 12.85 requests per vehicle-hour	Homewood's share of the vehicles; rebookings	Load per van: 5 vans × 12.85 → 64 requests at 6 pm (64 / 50 / 54 over 3 h)
Where trips go			
Pickup and drop-off totals per place, separately	A trip table matching both totals	Who travels where	Gravity prior (longer drives less likely, 0.1 per minute) + iterative fitting to both totals

Uncertain: the count is an illustrative range, ~150–260, not a confidence interval; a spread-out geography variant is built but not yet used in experiments

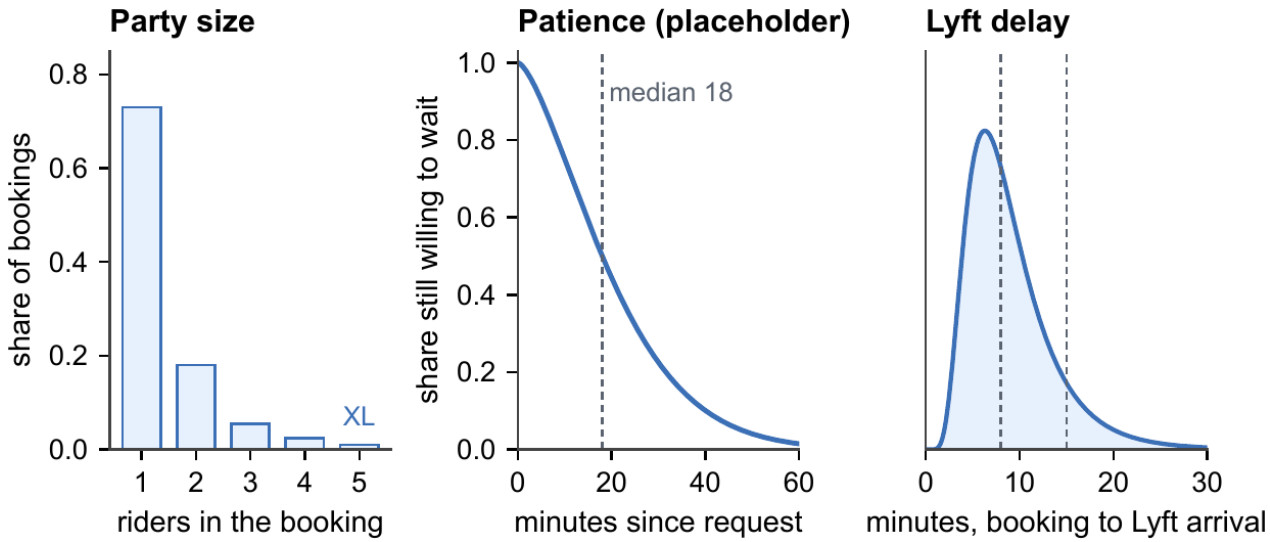
One synthetic night (L5, r00): 28 requests

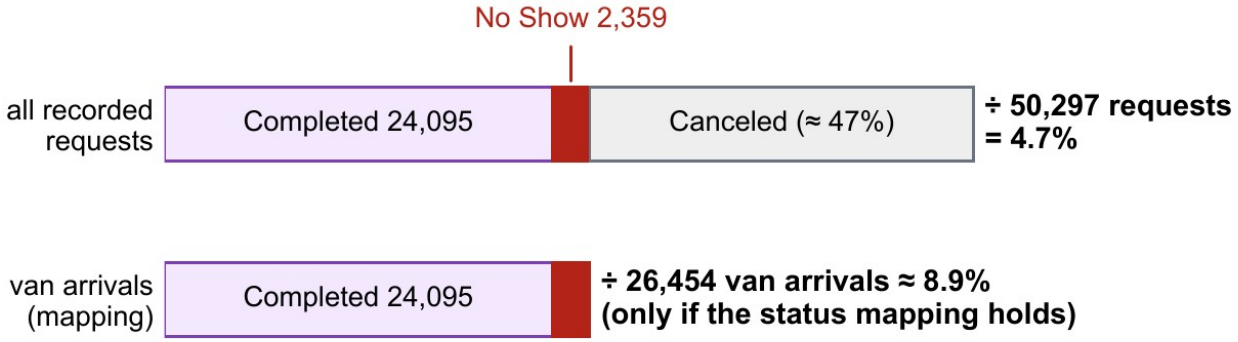


Lines pair each pickup with its drop-off, not road routes. Synthetic requests from the April-calibrated demand model.

Source: April dashboard aggregates via decision log D-35/D-36, D-18, D-49; methodology §4; figure: frozen L5 night r00.

Assumptions that complete the evening





DERIVED

Group size: 1.386 van passengers per completed van booking. That fixes a mean, not the distribution; ours is assumed (mean 1.405). A party of 5 needs a larger Lyft (XL).

ASSUMED

"Canceled" (≈47% of requests) mixes riders who gave up with requests handed to Lyft. Riders give up at a hidden personal deadline (placeholder: median 18 min); their share of "Canceled" is a provisional 50–75% sensitivity range.

DERIVED

No-shows: 4.7% of all requests, or about 8.9% of van arrivals only if the status labels mean what we assume. We use 8.9% per arrival, provisionally.

ASSUMED

Lyft: booked 2 min after the decision; arrives after a random delay (median 8, 90th percentile 15 min).

Source: Decision log D-38, D-39, D-40, D-43, D-48; trail §6–§9; methodology §4 and Appendix C.

How our algorithm operates, every time something happens

1 Reveal new requests; update vans and riders

↓

2 Generate feasible candidate routes per van

↓

3 MILP selects one route per van

↓

4 Lyft rule reads the predicted pickups

↓

5 Apply accepted transfers

↓

6 Re-plan once if anyone was transferred

↓

7 Execute: vans drive to their next stop

repeat at every request, cancellation, no-show and every 2 minutes

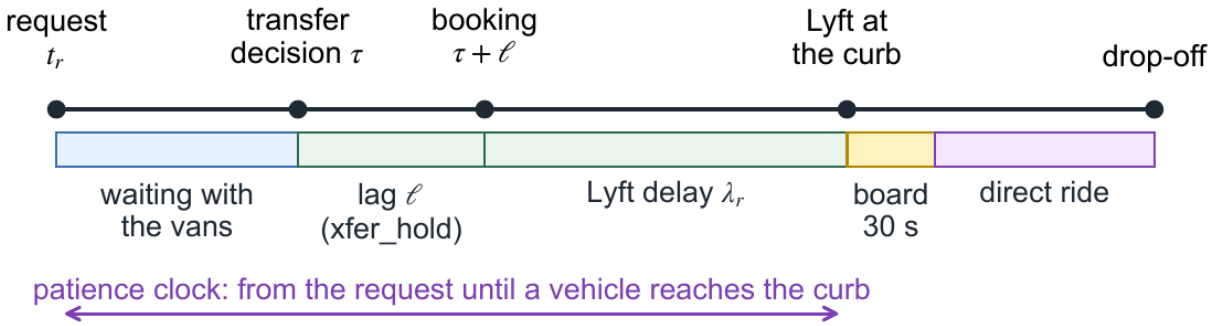
The dispatcher sees

- requests that have arrived
- where each van is, who is aboard
- travel times

Hidden until it happens

- each rider's patience deadline
- whether the rider will be at the curb
- each Lyft's actual delay

One actual transfer on a small designed evening (rule R2(15): transfer if predicted total wait > 15 min)



Rider R011: asked 18:16:30 · at 18:20 a van was predicted at 18:35:20 (total wait 18 min 50 s > 15) → transferred · booked 18:22 · Lyft at the curb 18:30 (wait 13 min 30 s) · dropped 18:33:25

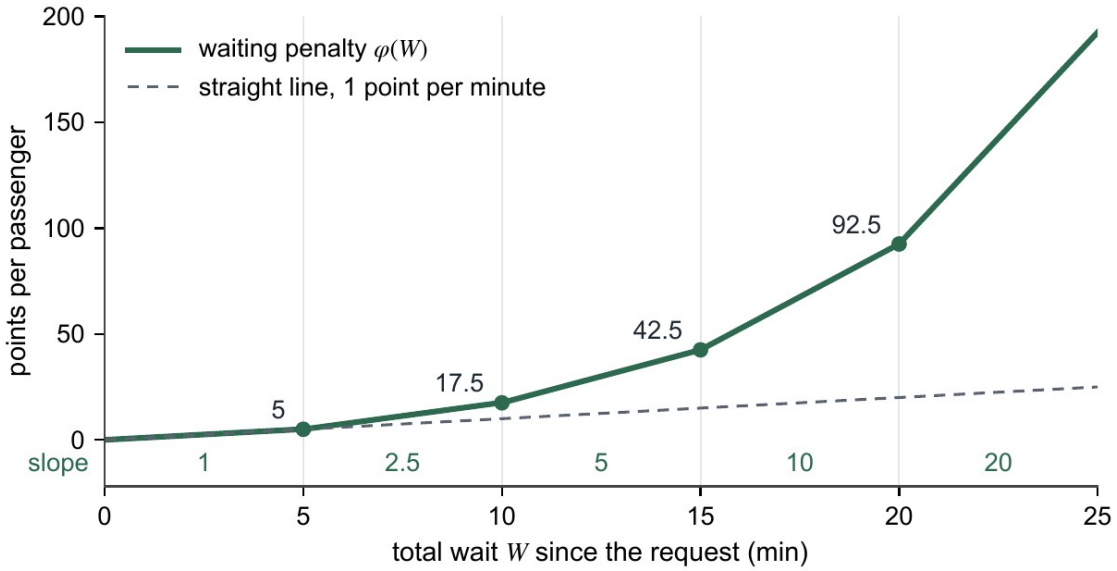
Source: scenario_spec.md v0.8 §17; decision log D-46, D-55; outputs/sim/m2_L4_report.md.

How the mathematics chooses a plan

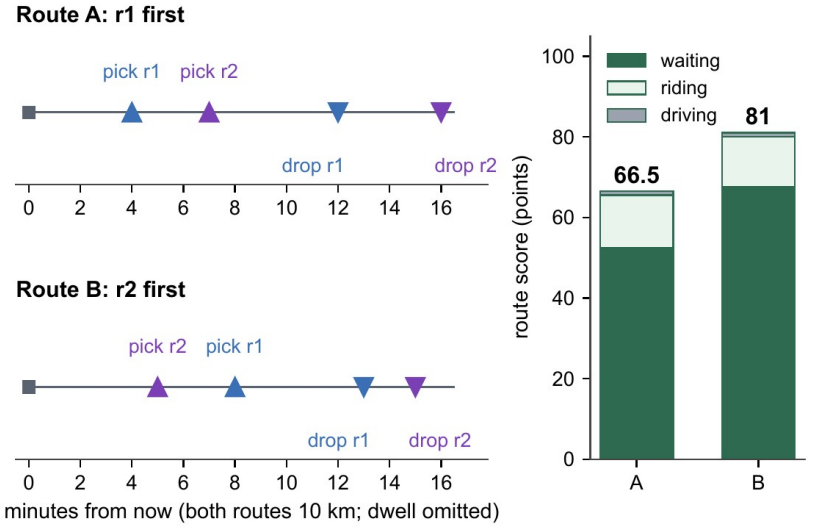
Each candidate route gets penalty points (not money); the MILP picks the cheapest compatible set

route points = Σ party × φ(wait) + 0.5 × Σ party × ride minutes + 0.1 × km

A request in no chosen route is left waiting at a price (slide 7); a large constant makes the model first serve as many requests as it can.



φ: each extra minute of waiting costs more the longer the rider has already waited (1, 2.5, 5, 10, 20 points per minute after 0, 5, 10, 15, 20 minutes)



One van; r1 (1 rider, waited 8 min) and r2 (2 riders, waited 1 min)

Route A (long waiter first): 66.5 points
Route B: 81.0 points → A is chosen

"Optimal" means optimal over the candidate routes generated at that moment (up to 3 new pickups per van), proven by the solver. Not optimal over all possible routes, and not over the whole evening.

Source: model_design.md v0.5 §3–§4; methodology §6 (worked example); decision log D-29, D-52.

A weakness we found, tested and addressed

Problem: the price for leaving a request waiting (the "pending price") assumed a van could reach it at the next 2-minute re-plan, sooner than any van physically could. So the cheapest requests to postpone were often those that had waited longest.

Same small designed evening (16 requests), whole evening	Original price	A: earliest physical pickup	B: after a chosen route
Wait median / 90th pct / max (min)	17.1 / 31.0 / 44.4	18.6 / 31.2 / 35.6	21.1 / 34.3 / 38.6
Mean wait (min)	18.8	18.8	21.8
Total wait of the party of five that had already waited 15 min by 18:15	22 min 18 s	19 min 28 s	20 min 39 s
Extra MILP variables / total solve time	0 / 0.1 s total	0 / 0.1 s total	up to 9,828 / 4.8 s total

Still an approximation: A ignores work the nearest van is already doing, so postponing can still look too cheap. The experiment checked behaviour over a whole evening: A cut the longest waits but redistributed waiting (the median rose and the most recent request waited longer); it did not improve every outcome.

Decision: A (approved, provisional)

- a direct physical pickup estimate
- no new variables
- keeps "serve as many as possible first"

Reason: clear meaning at no added cost, not that one rider was served sooner.

Source: Decision log D-53 (update 2, approval); outputs/sim/d53_experiment/d53_comparison.md; review note d53_pending_price_review.

How we know the code does what we specify

to the second

hand-computed answer keys reproduced on three designed evenings (L1–L3)

22 + 4

Lyft transfers and declined offers (there, riders accept with probability 0.9) matched on a fourth designed evening, summed over the five rules that transfer

292,181

plan combinations enumerated by an independent brute-force program (the designed evening under the original and the approved pending price); the MILP's choice matched at every epoch

253

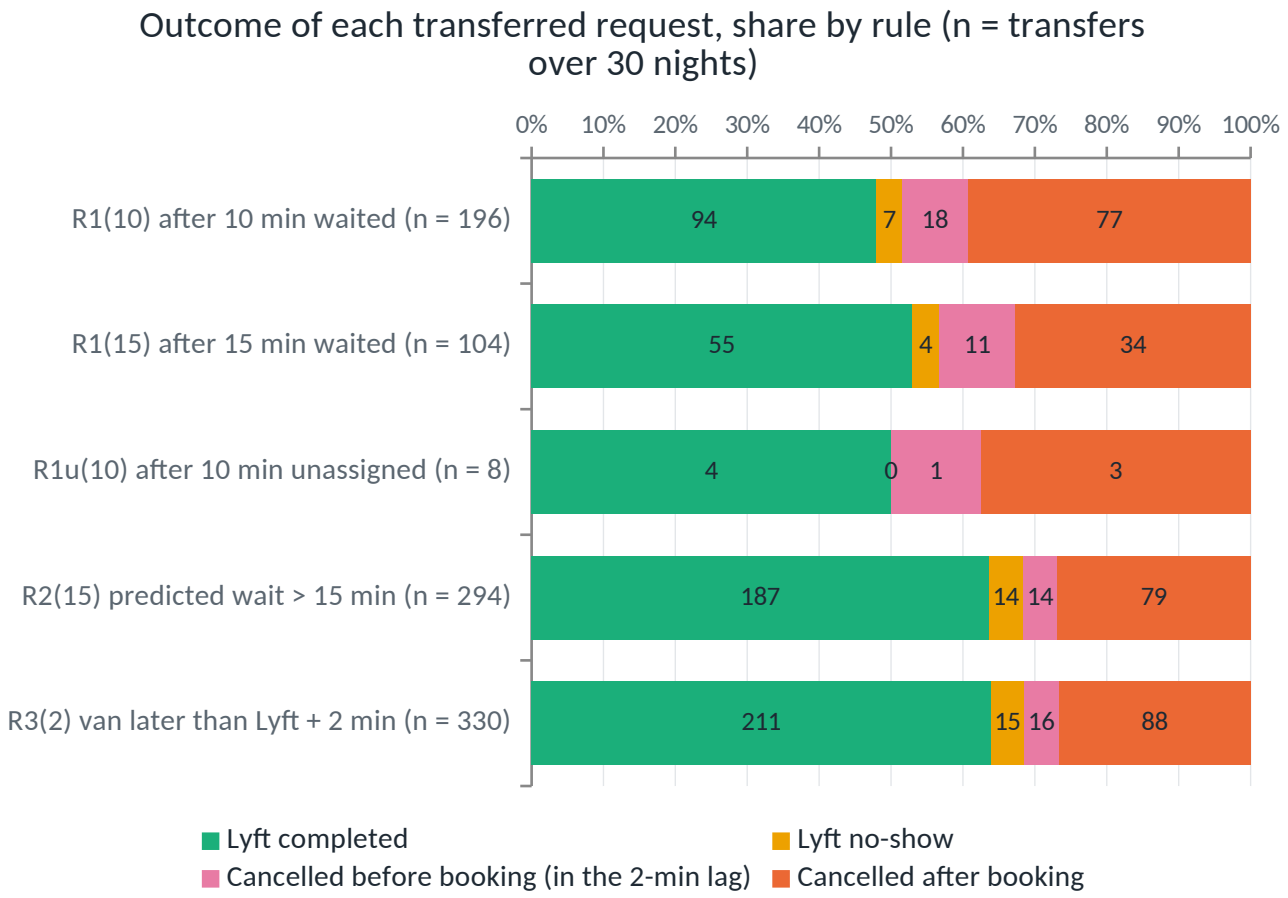
automated tests (207 on the simulator); 168 written by independent reviewers who did not reuse the simulator's logic

Plus on every run: one outcome per request, counts reconcile, seats and pickup-before-drop-off hold, inputs unchanged, sampled reruns identical. This checks the code against our rules, not our rules against the real service.

Source: Phase 4 log; decision log D-52, D-55, D-57; tests/ (test_sim_*, test_m2_independent, test_pending_price_independent).

Thirty synthetic nights: what the rules actually did

Pilot: 30 synthetic one-hour nights, each replayed under all six rules (same riders, same hidden traits). Two vans for 28 requests: a heavy load by design (14 per van-hour, a 10-minute burst). Provisional settings; the five-van evening is not yet run.



A transfer is not a rescue

Patience keeps running: 95 of 196 requests transferred under R1(10) cancelled before their Lyft arrived (77 after booking); about a third under R2(15) and R3(2).

Assigned is not picked up

"Transfer after 10 min with no van assigned" (R1u(10)) fired only when a request stayed unassigned that long: 8 transfers on 6 of 30 nights, all for requests made during the burst.

Rides over the limit

Plans never exceed the ride limit, but rides did 36 times across the 180 runs (16 distinct requests on 10 nights); each coincided with a no-show wait on that van while the rider was aboard: consistent with, not proof of, that cause.

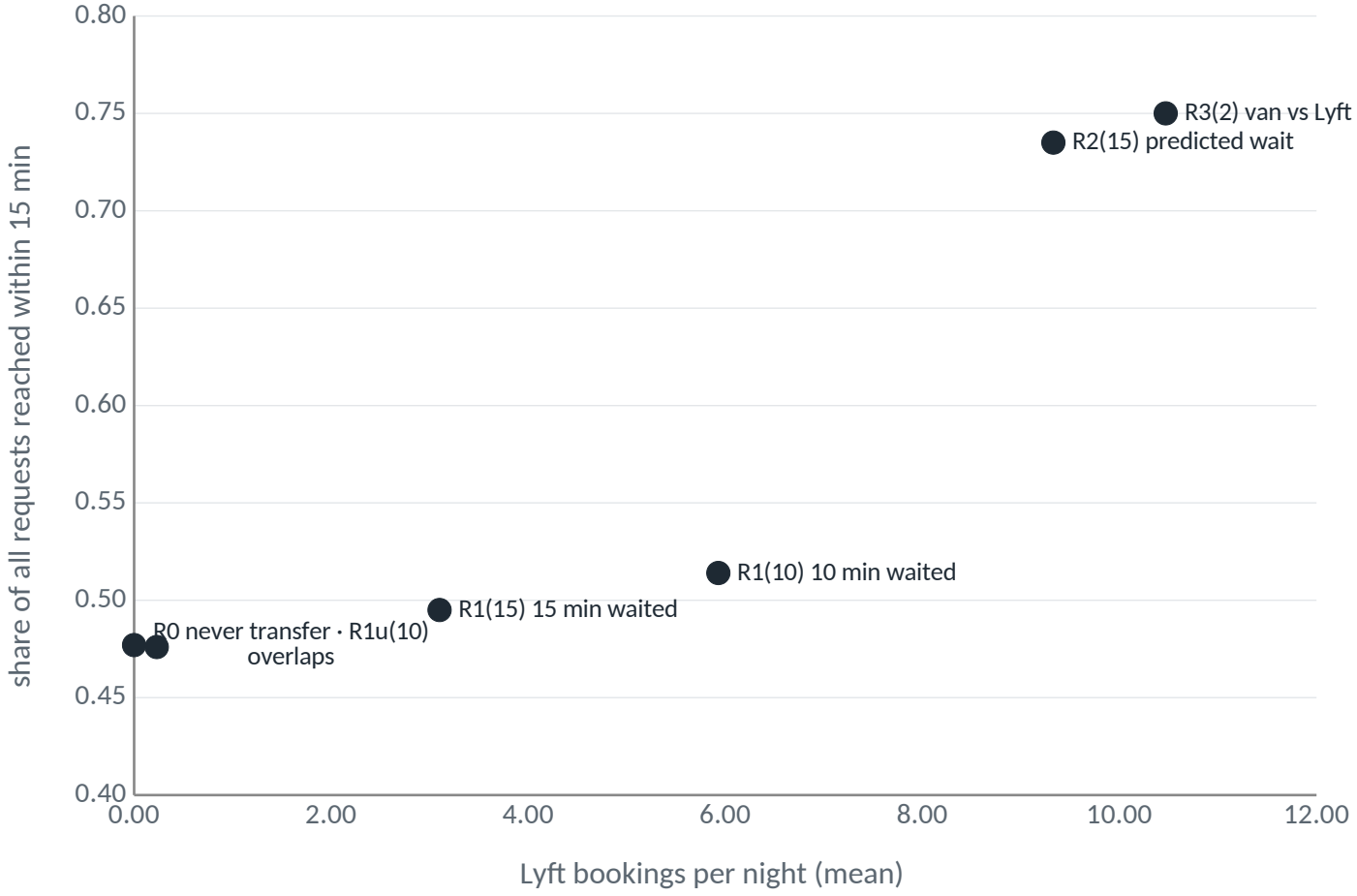
Mind the denominator

Wait percentiles cover only riders picked up (16 of 28 under never-transfer); "reached within 15 min" counts every request.

Source: outputs/sim/batch_L5_v1 (summary.md, per-run outcomes); decision log D-56, D-57. Counts are totals over 30 nights.

Service versus Lyft use: a trade-off among tested rules

Service = share of all requests a vehicle reached at the curb within 15 min. Rule P dominates Q if P has at least Q's service with no more Lyft bookings, and is strictly better in one.



Nondominated among the six tested

On the 30-night means: R0 (never transfer), R1(10) (after 10 min waited), R1(15) (after 15 min waited), R2(15) (predicted wait > 15 min), R3(2) (van later than Lyft + 2 min) are nondominated. R1u(10) (after 10 min unassigned) is technically dominated by R0, but the service gap (0.476 vs 0.477) is far inside the night-to-night spread (SD about 0.09): in practice indistinguishable from never transferring. Visible pattern: the two rules that read predicted pickups reach about 0.74 of requests, against 0.48–0.51 for the others, using about 9–10 Lyft bookings a night.

- Not a frontier of all possible policies
- Not a recommendation: picking a point needs a value judgement
- Weights inside the routing objective are a different trade-off
- Decision criteria (margins, intervals) are not yet agreed

Source: outputs/sim/batch_L5_v1/tradeoff.md (src/sim/tradeoff.py; descriptive, saved outputs only).

A leader–follower reading: possible, but not what we solved

Leader: policy designer

chooses the transfer rule and its threshold θ

Follower: routing optimizer

re-plans the vans around the riders left after transfers

Formal version (an extension):

$$\text{choose } \theta \text{ to minimise } \; \mathrm{E}[\, \text{service loss}(\theta, y) + \lambda \times \text{Lyft bookings}(\theta)\,]$$
$$\text{subject to } \; y \in \text{argmin over feasible routes of routing cost}(y;\ \text{riders not transferred under } \theta)$$

What we implemented

- a sequential policy simulation: plan → rule reads plan → transfers → re-plan once
- six fixed rules compared by replaying nights

Missing for a solved bilevel model

- an upper-level optimisation over θ
- the follower's response stated as constraints over the whole evening (ours is per epoch, restricted)
- a treatment of the expectation over nights

Riders follow assumed patience rules and Lyft is a delay distribution: they are not modelled as optimizing players.

Source: Bilevel and leader–follower terminology: Colson, Marcotte & Savard (2007); Dempe (2002); von Stackelberg (1934). Implementation: spec §17.4, D-55.

Limitations, next experiments and what we contribute

Limitations

- Aggregate data only; no comparison with real nights yet
- Patience is a placeholder; no-show probability provisional
- Results so far: two-van pilot; five-van evening not yet run
- Up to 3 new pickups planned per van; some runs take minutes
- One night pattern; designed evenings are small

Next

- Agree the decision criteria, then compute paired intervals on the saved 30 nights (to size the main experiment, not to conclude)
- Sweep rule thresholds to trace trade-off curves
- Build the five-van evening and check the MILP is fast enough there
- Test sensitivity to patience and Lyft delay

Intended contribution

A transparent, code-verified way to turn aggregate dashboards into paired synthetic experiments that say clearly what is observed, what is assumed and what the simulation produced. Not yet validated against real service nights.

Source: Phase 4 plan and log; decision log D-50, D-56, D-57; methodology §7–§9.

Backup slides

B1 MILP and bound · B2 event order and lifecycle · B3 code key · B4 parameter provenance · B5 all-rule batch table · B6 per-night trade-off · B7 pending-price formulas · B8 numerical diagnostics

B1 The route-selection MILP at one epoch

$$\min \ \Sigma_v \Sigma_k c_{vk} y_{vk} + \Sigma_r \pi_r u_r$$

$$\text{s.t.} \ \Sigma_k y_{vk} = 1 \quad \text{for every van } v \qquad \text{(one route per van)}$$

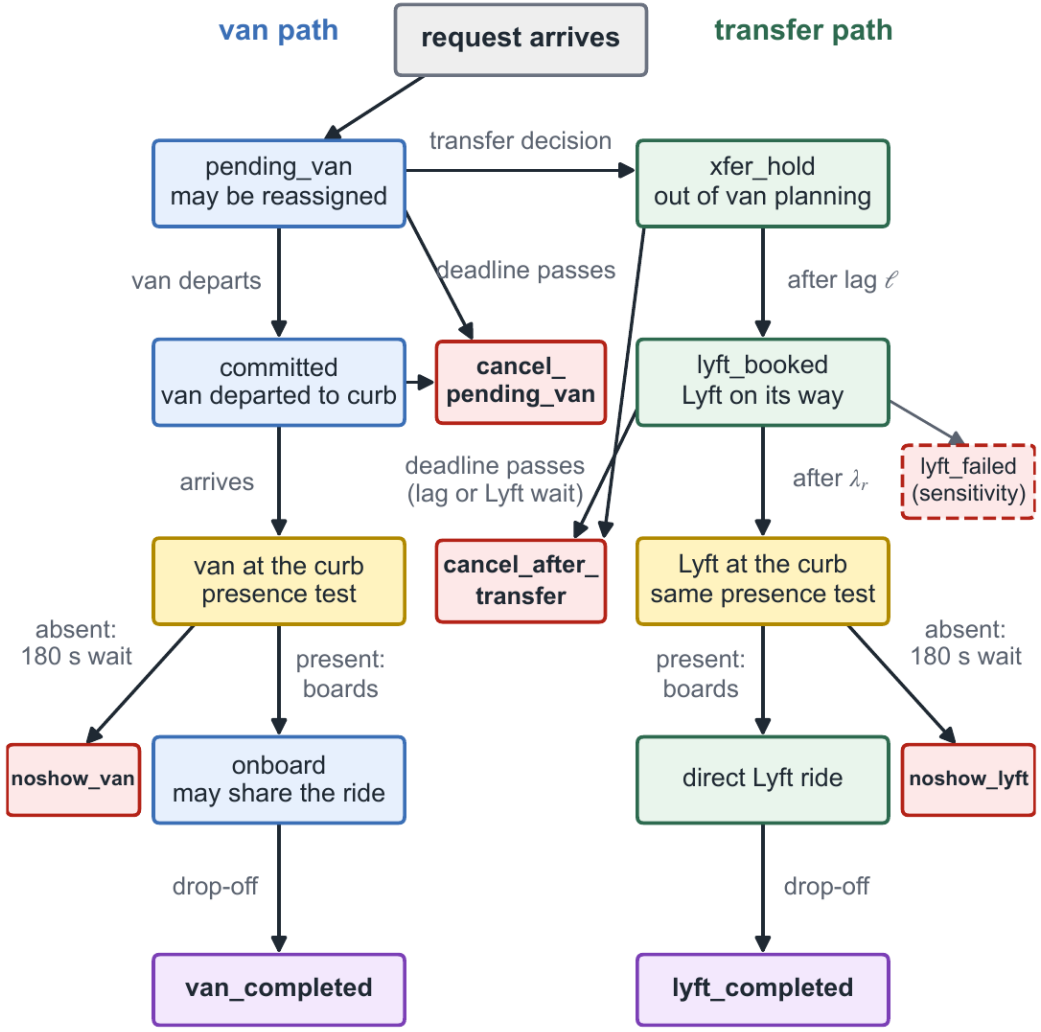
$$\Sigma_{\{v,k:\, r \in S_k\}} y_{vk} + u_r = 1 \quad \text{for every open request } r \quad \text{(served once, or pending)}$$

$$y, u \in \{0, 1\}$$

- c_{vk}: route points (waiting φ, riding 0.5/passenger-min, 0.1/km; Ψ only for unavoidable guardrail excesses); exact integers
- Candidates: every subset of ≤ 3 open requests per van, best order of all its stops (exact search); the van's committed-work route is always a candidate
- $\pi_r = b_r + M$ with b_r from pending price A; $M = 1 + \Sigma_v (\max_k c_{vk} - \min_k c_{vk}) + \Sigma_r b_r$ guarantees that every proven-optimal solution leaves the fewest requests pending (per request, not per passenger)
- HiGHS: relative gap 0, absolute gap 0.5 units, node limit 100,000; ties broken canonically; a failed solve is logged, never replaced by a heuristic

Source: model_design.md v0.5 §3; decision log D-47, D-52 (amendments), D-53 approval.

B2 Event order and a request's life



- Same-second order: drop-offs → curb arrivals (stop the patience clock) → service ends → vans on/off duty → new requests → patience checks → Lyft bookings → planning epoch → departures
- A van commits to a stop when it departs toward it
- During an absent rider's 3-min curb wait, planning assumes possible boarding at expiry; the simulator never boards an absent rider (D-54)
- Planning takes zero simulated time; wall-clock time is logged only

Source: scenario_spec.md v0.8 §17; decision log D-46, D-54.

B3 Code key

Code	Plain meaning
R0	Never transfer to Lyft (reference)
R1(10), R1(15)	Transfer after 10 or 15 minutes of waiting
R1u(10)	Transfer after 10 minutes with no van assigned in the plan
R2(15)	Transfer if the predicted total wait exceeds 15 minutes
R3(2)	Transfer if the van is predicted later than a Lyft could arrive, plus 2 minutes
L1–L4	Small designed evenings with hand-computed answer keys (L4: Lyft bookkeeping, vans from 18:15)
L5	Pilot: 30 synthetic one-hour nights, 2 vans, 28 requests each
C	Our van dispatcher: a restricted route-selection MILP solved with HiGHS
A	Pending price: a waiting request is charged its earliest physically possible pickup
D-xx	Numbered entry in the decision log
MILP	Mixed-integer linear program
XL	Larger Lyft vehicle, used for a party of 5

Source: lyft.py; scenario_spec.md §6, §8; decision log.

B4 Where each input comes from

Input	Status	Value	Basis
Requests per hour	DERIVED	64 / 50 / 54 for five vans (pilot: 28 in one hour, two vans)	D-35, D-36; L5 design
Trip pairing	ASSUMED + fitted	Gravity prior, decay 0.1/min, fitted to both April margins	D-18
Group size	ASSUMED (mean derived)	P(1..5) = .73/.18/.055/.025/.01; April 1.386 van passengers per completed van booking	D-38
Patience	ASSUMED placeholder	Weibull, shape 1.5, median 18 min	D-39
No-show	DERIVED, conditional	8.9% per arrival if status mapping holds (4.7% of all requests)	D-48
Lyft	ASSUMED	Lag 2 min; delay lognormal, median 8, 90th pct 15 min	D-43
Travel times	OBSERVED (routing)	Local OSRM road network; van × 1.2 in the pilot	D-31, D-47

Source: decision log D-18–D-48; methodology §4.

B5 All six rules over 30 synthetic nights

Per night	R0 never transfer	R1(10) after 10 min waited	R1(15) after 15 min waited	R1u(10) after 10 min unassigned	R2(15) predicted wait > 15 min	R3(2) van later than Lyft + 2 min
Reached within 15 min (share of 28)	0.48 (0.09)	0.51 (0.08)	0.50 (0.09)	0.48 (0.10)	0.73 (0.07)	0.75 (0.08)
Requests picked up (of 28)	16.1 (2.4)	17.2 (2.6)	16.5 (2.6)	16.2 (2.5)	20.2 (2.0)	20.4 (2.3)
Wait median, min (picked up)	8.6 (1.8)	8.3 (1.6)	8.4 (1.5)	8.9 (2.1)	7.5 (1.0)	7.3 (1.0)
Wait 90th pct, min (picked up)	20.2 (5.3)	18.1 (2.4)	19.6 (4.0)	20.2 (4.6)	13.3 (1.5)	12.2 (1.7)
Wait 90th pct, min (passengers)	20.1 (5.8)	17.4 (2.7)	19.2 (4.8)	19.6 (4.7)	13.2 (2.3)	12.7 (2.7)
Van completed	16.1 (2.4)	14.1 (2.3)	14.6 (2.4)	16.1 (2.5)	14.0 (2.7)	13.3 (2.6)
Lyft completed	0.0 (0.0)	3.1 (1.4)	1.8 (1.2)	0.1 (0.3)	6.2 (1.9)	7.0 (1.6)
Cancelled waiting for van	10.8 (2.3)	6.5 (2.3)	8.9 (2.5)	10.5 (2.4)	3.3 (1.7)	2.6 (1.5)
Cancelled after transfer	0.0 (0.0)	3.2 (1.8)	1.5 (1.0)	0.1 (0.4)	3.1 (1.5)	3.5 (1.5)
Lyft bookings	0.0 (0.0)	5.9 (1.6)	3.1 (1.4)	0.2 (0.5)	9.3 (2.5)	10.5 (1.9)
Van-km	51.2 (4.4)	47.6 (5.5)	48.6 (5.0)	51.9 (4.8)	45.7 (5.7)	43.0 (4.0)
Run time, s	43 (46)	40 (44)	49 (57)	50 (56)	5 (7)	4 (5)

Cells: mean of nightly values (standard deviation across nights). Waits cover only requests picked up.

Source: outputs/sim/batch_L5_v1/summary.md.

B6 Trade-off night by night

Rule	Reach (mean)	Lyft bookings (mean)	Nights nondominated	Nights = R0	Reach above / below R0
R0 never transfer	0.477	0.00	30 of 30	30	0 / 0
R1(10) after 10 min waited	0.514	5.93	11 of 30	0	19 / 6
R1(15) after 15 min waited	0.495	3.10	14 of 30	1	15 / 5
R1u(10) after 10 min unassigned	0.476	0.23	26 of 30	24	2 / 2
R2(15) predicted wait > 15 min	0.735	9.33	24 of 30	0	30 / 0
R3(2) van later than Lyft + 2 min	0.750	10.47	12 of 30	0	30 / 0

A rule that makes no transfer on a night replays never-transfer exactly and ties it. Nights with reach above / below never-transfer are paired comparisons of the same synthetic night.

Source: outputs/sim/batch_L5_v1/tradeoff.md.

B7 The three pending prices

- Original: $b_r = q_r \cdot \varphi(\text{next tick} - t_r)$
- A (approved, provisional): $b_r = q_r \cdot \varphi(\max(\text{next tick}, L_r) - t_r)$, L_r = earliest any on-duty van can reach the pickup
- B (experimental): price after a chosen van's route, linearised with continuous variables $z_{rj} \le y_j$, $\Sigma_j z_{rj} = u_r$
- A is a lower bound on any future service (up to a 13-s matrix exception, vans not yet on duty, the next-tick floor); B can err in both directions and adds $|\text{requests}| \times |\text{routes}|$ variables
- Neither is a guaranteed feasible joint schedule for all waiting riders

Source: model_design.md v0.5 §3.5; decision log D-53; research_notes/d53_pending_price_review_2026-10-22.md.

B8 Numerical diagnostics (L5 pilot)

- 180 runs = 30 synthetic nights × 6 rules; every run passed every check; one code version for all runs
- 13,663 planning epochs; main solve proven optimal in all (none unproven, none failed)
- 41 epochs had an incomplete tie search (a solver error while looking for equally good plans); in all 41 a full enumeration shows a unique optimum, which is the plan chosen
- 36 ride-guardrail excesses across the 180 runs (16 distinct requests on 10 nights, repeated across rules), each coinciding with a no-show wait on that van while the rider was aboard (attribution check, not proof); 0 unexplained
- Run time per run: mean 32 s, longest 253 s (Python, one core per run)
- Reproducibility: 6 of 6 sampled reruns identical byte for byte; the 2 night-r00 batch runs identical to the earlier pilot outputs

Source: outputs/sim/batch_L5_v1 (summary.md, tie_incomplete_check.csv, reproducibility.json); decision log D-52 am. 5, D-57.